

**Subject Name: OBJECT ORIENTED PROGRAMMING DESIGN**

**Subject Code: CS T33**

**2 MARKS AND 11 MARKS**

**UNIT III**

---

**Pointers and arrays:** Pointer to class and object – pointer to derived classes and base classes – accessing private members with pointers – address of object and void pointers – characteristics of arrays – array of classes.

**Memory:** Memory models – The new and delete operators – Heap consumption – Overloading new and delete operators – Execution sequence of constructors and destructors – specifying address of an object – dynamic objects.

**Binding, Polymorphism and Virtual Functions:** Binding in C++ – Pointer to derived class objects – virtual functions – Array of pointers – Abstract classes – Virtual functions in derived classes – constructors and virtual functions – virtual destructors – destructors and virtual functions. Strings - Declaring and initializing string objects – relational operators – Handling string objects – String attributes – Accessing elements of strings – comparing and exchanging and Miscellaneous functions.

---

**PART A – 2 MARK QUESTIONS**

|                                                             |
|-------------------------------------------------------------|
| <b>1. What is meant by dynamic binding or late binding?</b> |
|-------------------------------------------------------------|

Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at the run-time.

|                                      |
|--------------------------------------|
| <b>2. What are abstract classes?</b> |
|--------------------------------------|

Classes containing at least one pure virtual function become abstract classes. Classes inheriting abstract classes must redefine the pure virtual functions; otherwise the derived classes also will become abstract. Abstract classes cannot be instantiated.

**3. Define virtual function.**

Virtual functions allow programmers to declare functions in a baseclass, which can be defined in each derived class. A pointer to an object of a base class can also point to the objects of its derived class.

**4. Define Pure Virtual Function.**

Pure virtual function is declared as a function with its declaration followed by = 0. Virtual functions are defined with a null body. It has no definition. These are similar to do nothing or dummy function

**5. What are the properties of pure virtual function?**

- \* A pure virtual function has no implementation in the base class.
- \* A class with pure virtual function cannot be instantiated.
- \* It acts like an empty bucket that the derived class is supposed to fill.
- \* A pure virtual member function can be invoked by its derived class.

**6. What is the syntax used for writing pure virtual function?**

```
classmyclass
{
    public:
        virtualreturntypefunctionname(args) = 0 ;
}
```

**7. Define polymorphism.**

Polymorphism means the ability to take more than one form. That means it performs more than one operation by using a single operator. Polymorphism is implemented using the overloaded functions and operators.

**8. Define pointer to an object.**

Pointer to an object is a variable containing an address of an object. It is similar to a pointer to any other variable.

Syntax:

Classname \*ptr-to-object;

Ptr-to-object = &object;

Ptr-to-object = new classname;

**9. Define this pointer.**

This pointer is a pointer to an object invoked by the member function. Thus member function of every object has access to a pointer named “this”, which points to the object itself.

**10. Define pointers to member**

It is possible to take the address of a member of a class and assign it to a pointer. The address of a member can be obtained by applying the operator & to a “fully qualified” class member name. A class member pointer can be declared using the operator :: \* with the class name.

Eg: class A

{

int m;

public:

void show( );

};

pointer to member m is defined as

int A::\*ip=&A::m;

A::\*->pointer to member of A class

&A::m->address of the m member of A class

**11. What are the types of polymorphism?****Basic Types**

i)Runtime polymorphism

ii)Compile time polymorphism

**Major Types**

a.Run-time polymorphism

b.Compile time polymorphism

c.ad-hoc polymorphism

d.Parametric polymorphism

e.Virtual functions

f.Function name overloading

g.Operator overloading

**12. What is compile time polymorphism?**

The overloaded member functions are selected for invoking by matching arguments both type and number. This information is known to the compiler at the compile time and therefore compiler is able to select the appropriate function for a particular call at the compile time itself. This is called early binding or static binding or static linking. Also known as compile time polymorphism.

**13. List the features of pointers.**

- Pointers save the memory space
- Execution time with pointer is faster because data is manipulated with the address.
- The memory is accessed efficiently with the pointers. The pointer assigns the memory space dynamically that leads to reserve specific bytes in the memory.
- Pointers are used with data structures. They are useful for representing two-dimensional and multi-dimensional arrays.

**14. Define array.**

Array is a collection of elements of similar data types in which each element is unique and located in separate memory locations. Declaration of an array is given below:

```
int a[5];
```

it tells the compiler that 'a' is an integer type of array and it must store five integers.

**15. Write the characteristics of arrays.**

- The declaration `inta[5]` is creation of five variables of integer type in memory. Instead of declaring five variables for five values, the programmer can define them in an array.
- All the elements of an array share the same name, and they are distinguished from one another with the help of element number.
- The element number in an array plays major role in calling each element.
- Any particular element of an array can be modified separately without disturbing other elements.
- Any element of an array `a[]` can be assigned/equated to another ordinary variable or array variable of its type.

**16. What is void pointer?**

Pointers can also be declared as void type. Void pointers cannot be dereferenced without explicit type conversion. This is because being void the compiler cannot determine the size of the object the pointer points to.

**17. What is the advantage of using dynamic initialization?**

The advantage of using dynamic initialization is that various initialization formats can be provided using overloaded constructor.

**18. What are the rules for virtual function?**

1. They cannot be static members
2. They are accessed by using object pointers
3. A virtual function can be a friend of another class.

**19. What are the types in compile time polymorphism?**

1. Function overloading
2. Operator overloading

**20. Explain overloading of new and delete operators?**

The memory allocation operators `new` and `delete` can be overloaded to handle memory resource in a customized way. The main reason for overloading these functions is to increase the efficiency of memory management.

**21. Write the difference between `new` and `malloc()`**

| <b>new</b>                                           | <b>malloc()</b>                            |
|------------------------------------------------------|--------------------------------------------|
| Creates objects                                      | Allocates memory                           |
| Returns pointer of relevant type                     | Returns void pointer                       |
| It is possible to overload <code>new</code> operator | <code>malloc()</code> cannot be overloaded |

**22. List the string manipulating functions.**

| <b>Function</b>         | <b>Use</b>                                               |
|-------------------------|----------------------------------------------------------|
| <code>append()</code>   | Adds one string at the end of another string             |
| <code>assign()</code>   | Assigns a specified part of string                       |
| <code>at()</code>       | Access the characters located at given location          |
| <code>begin()</code>    | Returns a reference to the beginning of a string         |
| <code>capacity()</code> | Calculates the total elements that can be stored         |
| <code>compare()</code>  | Compares two strings                                     |
| <code>empty()</code>    | Returns false if the string is not empty, otherwise true |
| <code>end()</code>      | Returns a reference to the termination of string         |

**23. List the string manipulating operators.**

| Operator | Working                      |
|----------|------------------------------|
| =        | Assignment                   |
| +        | Joining two or more strings  |
| +=       | Concatenation and assignment |
| ==       | Equality                     |
| !=       | Not equal to                 |
| <        | Less than                    |
| <=       | Less than or equal           |
| >        | Greater than                 |
| >=       | Greater than or equal        |
| []       | Subscription                 |
| <<       | Insertion operator           |
| >>       | Extraction operator          |

**24. What is meant by string?**

A string is a sequence of characters. They are declared as character arrays. Each element of string occupies a byte in the memory. Every string is terminated by a null character.

It can be declared as

```
char text[10];
```

**25. Write a program to remove the specified characters from string.**

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1("abc12345defg");
    cout<<"\n s1 ="<<s1;
```

```
        cout<<"\n after erase()";  
        s1.erase(3,5);  
        cout<<"\n s1 ="<<s1;  
        return 0;  
    }
```

### **OUTPUT**

```
s1 = abc12345dfeg  
after erase()  
s1 = abcdefg
```

### **PART-B(11 MARKS)**

|                                                          |
|----------------------------------------------------------|
| <b>1. Explain with an example of pointer to a class.</b> |
|----------------------------------------------------------|

A pointer is a variable that holds the address of another data variable. The variable may be of any data type i.e., int, float or double. In the same way, we can also define pointer to class. Here, starting address of the member variables can be accessed. Such pointers are called class pointers.

**//Example program**

```
#include<iostream.h>
#include<conio.h>
class man
{
public:
char name[10];
int age;
};
void main()
{
man m={"ravindra",15};
man *ptr;
ptr=&m;
clrscr();
cout<<"\n Name ="<<m.name<<"\n Age ="<<m.age;
cout<<"\n Name ="<<ptr->name<<"\n Age ="<<ptr->age;
}
```

**Output**

```
Name=ravindra
Age=15
Name=ravindra
Age=15
```

**Explanation:** In the above program pointer ptr points to the object m. The statement ptr=&m; assigns address of first member element of the class to the pointer ptr. Using the dot operator and arrow operator, contents can be displayed. The display of class contents is possible because the class variables are declared as public. The statement below can also be used to assign address of object to the pointer.

- ptr=&(man)m;
- \*ptr=(man)m;
- \*ptr=man(n);
- \*ptr=m;
- ptr=&m;

|                                                                              |
|------------------------------------------------------------------------------|
| <p><b>2. Briefly explain the concept pointer to object with example.</b></p> |
|------------------------------------------------------------------------------|

Like variables, objects also have an address. A pointer can point to specified object. The following program illustrates this.

**Example program 1: write a program to declare an object and pointer to the class. Invoke member functions using pointer.**

```
#include<iostream.h>
#include<conio.h>
class Bill
{
public:
float price;
float amount;
int qty;
void getdata(int a,float b,float c)
{
qty=a;
price=b;
amount=c;
}
void show()
{
cout<<"quantity:"<<qty;
```

```
cout<<"\n price:"<<price;
cout<<"\n amount:"<<amount;
}
};
void main()
{
clrscr();
Bill s;
Bill *ptr=&s;
ptr->getdata(45,10.0,45*10.0);
(*ptr).show (); //ptr->show ();
}
```

**Output:**

```
Quatity: 45
Price: 10.0
Amount: 450
```

**Explanation:**

In the above program, the class Bill also contains member functions grtdata() and show() to read and display the data. In function main(), s is an object of class Bill and ptr is pointer to the same class. The address of object s is assigned to pointer ptr. Using pointer ptr with arrow operator (→) and dot operator (.), member functions are invoked.

**Example program 2: write a program to create dynamically an array of objects of class type. Use new operator.**

```

#include<iostream.h>
#include<conio.h>
class Bill
{
public:
float price;
float amount;
int qty;
void getdata(int a,float b,float c)
{
qty=a;
price=b;
amount=c;
}
void show()
{
cout<<"quantity:"<<qty;
cout<<" \n price:"<<price;
cout<<"\n amount:"<<amount;
}
};

void main()
{
clrscr();
Bill *s=new Bill[2];
Bill *d=s;
int x,i;
float y;
for(i=0;i<2;i++)
{
cout<<"\n Enter quantity and price:";
cin>>x>>y;
s->getdata(x,y,x*y);
s++;
}
for(i=0;i<2;i++)
{
cout<<endl;
d->show();
d++;
}
}

```

Output:

```

Enter quantity and price: 5 5.3
Enter quantity and price: 8 9.5
quantity: 5
price: 5.3
amount: 26.5

quantity: 8
price: 9.5
amount: 76

```

**Explanation :**

In main(), using new memory allocation operator, memory required for two objects is allocated to pointer s i.e., 10 bytes. The first for loop accepts the data through the keyboard. Immediately after this, the data is sent to the member function getdata(). The second for loop is used to display the content on the screen. Here, the function show() is invoked.

**3. Explain the concept of pointer to derived classes and base classes with example.**

It is possible to declare a pointer which points to the base class as well as the derived class. One pointer can point to different classes. For example, x is a base class and y is a derived class. The pointer pointing to x can also point to y.

**Example : write a program to declare a pointer to the base class and access the member of base and derived class.**

```
#include<iostream.h>
#include<conio.h>
class A
{
public:
int b;
void display()
{
cout<<"b="<<b;
}
};

class B:public A
{
public:
int d;
void display()
{
cout<<"\n b="<<b;
}
};

void main()
{
clrscr();
A *cp;
A base;
cp=&base;
cp->b=100;
//cp->d=200; not accessible
cout<<"\n cp points to the base objects:";
cp->display();
Bb;
cout<<"\n cp points to the derived class:";
cp=&b;
cp->b=150;
//cp->d=300; not accessible
cp->display();
}
```

**Output:**

```
cp points to the base objects
b=100
cp points to the derived class
b=150
```

**Explanation:** In the above program, A and B are two classes with one integer member variable and member function. The class B is derived from class A. The pointer cp to the class. The variable base is an object of the class A. the address of object is assigned to pointer cp. The pointer cp can access the member variable b, a member of base class but cannot access variable d, a member of derived class. Thus, the following statement is invalid.

```
cp->d=200;
```

the variable b is an object of class B. the address of object b is assigned to the pointer cp. However, b is an object of derived class and access to member variable d is not possible. Thus, the following statement is invalid.

```
cp->d=300;
```

Here, cp is pointer to the base class and could not access the member of derived class.

**4. Explain the concept of accessing private member with pointers.**

The public and private members of a class can be accessed using pointers. The public and private member variables are stored in successive memory locations. The following program explain how private member can also be accessed using pointer.

**//program to access members like public members of the class using pointers**

```
#include<iostream.h>
#include<conio.h>
class A
{
private:
int j;
public:
int x;
int y;
A()
{
j=20;
}
};

void main()
{
clrscr();
A a;
int *p;
a.x=11;
a.y=12;
p=&a.x;
p--;
cout<<"\n j="<<*p;
p++;
cout<<"\n x="<<*p;
p++;
cout<<"\n y="<<*p;
}
```

**OUTPUT:**

```
j=20
x=11
y=12
```

**Explanation:** In the above program the class A is declared with one private variable and two public variables. In main(), a is an object of class A and p is an integer pointer. The member variables are initialized with 11 and 12 using assignment operation.

The address of variable x is assigned to a pointer. The postfix decrementation of p gives the memory location of j, and value stored at those locations are accessed and displayed. The postfix incrementation operation gives the successive memory locations of x and y, and value stored at those locations are accessed and displayed.

### 5. Explain the concept of address of object and Void pointers

The size of object is equal to number of total member variables declared inside the class. The size of member function is not considered in object. The object itself contains address of first member variable. The address of object cannot be assigned to the pointer of the same type. This is because increase operation on pointers will set the address of object according to its size (size of object = size of total member variables). The address of object should be increased according to the basic data type. To overcome this situation a void pointer is

useful. The type of void pointer can be changed at run-time using typecasting syntax. Thus all the member variables can be accessed directly.

**Program to declare void pointers and access member variables using void pointers.**

|                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#include&lt;iostream.h&gt; #include&lt;conio.h&gt; #include&lt;stdio.h&gt; class A { protected: int x; int y; }; class B : public A { public: int z; B() { x = 10; y= 20; z= 30; } };</pre> | <pre>void main() { clrscr(); // clears the screen B b;      // object declaration int j;     // declaration of integer j void *p = &amp;b; // declaration &amp; initialization of void pointer for(j=0;j&lt;3;j++) // for loop executed for three times printf("\n member variable [%d] = %d", j+1, *((int*)p+j)); }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**OUTPUT**

```
member variable[1] = 10
member variable[2] = 20
member variable[3] = 30
```

**Explanation:**

In the above program, b is an object of derived class B. the address of object b is assigned to void pointer p. using for loop and typecasting operation, integer values are accessed. The address of void pointer is increased by two bytes. The cout statement now and then will not display the result properly and hence, the printf() statement is used. The typecasting operation is done according to the sequence of the member variables.

**6. Define arrays. Explain its characteristics.**

**Arrays** – Array is a collection of elements of similar data types in which each element is unique and located in separate memory locations.

**Characteristics of arrays**

- The declaration `int a[5]` is creation of five variables of integer type in memory. Instead of declaring five variables for five values, the programmer can define them in an array.
- All the elements of an array share the same name, and they are distinguished from one another with the help of element number.
- The element number in an array plays major role in calling each element.

- Any particular element of an array can be modified separately without disturbing other elements. `int a[5]={1,2,3,4,8}`; if the programmer needs to replace 8 with 10, it is not required to change all other numbers except 8. To carry out this task the statement `a[4]=10` can be used. Here the other three elements are not disturbed.
- Any element of an array `a[]` can be assigned/equated to another ordinary variable or array variable of its type.

**For example**

`b = a[2];`

`a[2] = a[3];`

- In the statement `b = a[2]` or vice versa, value of `a[2]` is assigned to `b`, where `b` is an integer.
  - In the statement `a[2] = a[3]` or vice versa, value of `a[2]` is assigned to `a[3]`, where both the elements are of the same array.
- The array elements are stored in continuous memory locations. The amount of storage required for holding elements of the array depends upon its type and size. The total size in bytes for a single dimensional array is computed as shown below.
    - **Total bytes = sizeof(data type) \* size of array**
  - The rules for array declaration are the same in C and C++. The only difference is the declaration of strings i.e character array.

| Program executed in C                                                                                                                                                              | Program executed in C++                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#include&lt;stdio.h&gt;  #include&lt;conio.h&gt;  void main()  {  char text[3] = “xyz”;  char txt[3] = “abc”;  clrscr();  printf(“\n%s”, text);  printf(“%s”, text);  }</pre> | <pre>#include&lt;stdio.h&gt;  #include&lt;conio.h&gt;  void main()  {  char text[3] = “xyz”;  char txt[3] = “abc”;  clrscr();  printf(“\n%s”, text);  printf(“%s”, text);  }</pre> |
| <b>OUTPUT</b>                                                                                                                                                                      | <b>OUTPUT</b>                                                                                                                                                                      |

xyz\_abcW\_

xyzA

**7. Explain in detail about array of classes?**

Array of classes can be declared as below:

```
class stud
{
public:
char name[12];      // class declaration
int rollno;
char grade[2];
};
```

```
class stud st[3];    // declaration of array of class object
```

In the above example, st[3] is an array of three elements containing three objects of class stud. Each element of st[3] has its own set class member variables.

**Program to create an array of class objects**

```
#include<stdio.h>
#include<conio.h>
#include<iostream.h>
void main()
{
int k = 0;
class stud
{
public:
char name[12];      // class declaration
int rollno;
char grade[2];
};

class stud st[3];
while(k<3)
{
clrscr();
gotoxy(2,4);
cout<<"Name:";
gotoxy(17,4);
cin>>st[k].name;
gotoxy(2,5);
cout<<"Roll No.:";
gotoxy(17,5);
cin>>st[k].rollno;
gotoxy(2,6);
cout<<"Grade:";
gotoxy(17,6);
cin>>st[k].grade[1]='\0';
puts("press any key...");
getch();
k++;
}
k = 0;
clrscr();
cout<<"\n Name\tRollno\tGrade\n";
while(k<3)
{
cout<<st[k].name<<"\t"<<st[k].rollno<<"\t"<<st[k].grade<<"\n";
k++;
}
}
```

**OUTPUT**

```
Name Rollno Grade
Balaji  50    A
Manoj   51    B
```

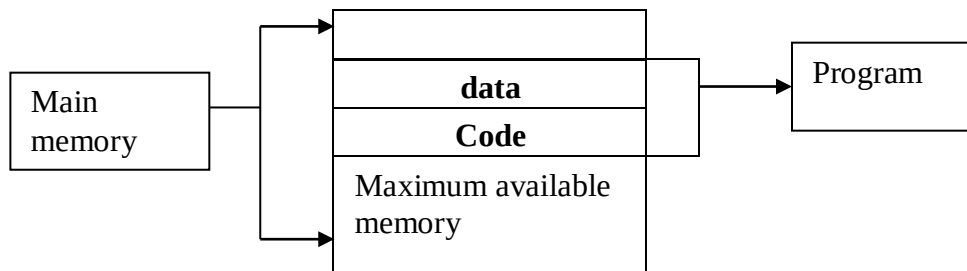
Sanjay 55 C

**Explanation:** In the above program, class stud is declared. Its members are char name[12]; int rollno and char grade[2] and all are public. The array st[3] of class stud is declared. The first while loop and cin() statements within the first while loop are used for repetitive data reading. The second while loop and printf() statements within it display the contents of array.

## 8. Explain in detail about memory models

### Memory models

- It sets the supportable size of code and data.
- Using memory model able to set the size limits of the code and data.
- C/C++ programs always use different segments for code and data.
- Memory model decides the default method of memory addressing.



### Tiny

- Use this model for average size programs.
- Near pointers are always used.
- Use the tiny model when memory is at an absolute premium.
- All four segment registers (CS,DS,ES,SS) are initialized with same address and all addressing is accomplished using 16 bits.
- Total memory capacity in this case is 64 K bytes.
- Programs are executed quickly if this model is selected.
- Near pointers are always used.
- Tiny model programs can be converted to .COM format.

### Small

- All code should fit in a single 64KB segment and all data should fit in a second 64 KB segment.
- All pointers are 16 bits in length.
- Execution speed is same as tiny model.

### Medium

- All data should fit in a single 64 KB segment.

- The code is allowed to use multiple segments.
- All pointers to data are 16 bits, but all jumps and calls require 32 bit addresses.
- Access to data is fast, slow in program execution.
- This model is for big programs that do not keep a large amount of data in memory.
- Far pointers are used for code but not for data.

**Compact**

- All code should fit in 64KB segment but the data can use multiple segments.
- All pointers to data are 32 bits, but jumps and calls can use 16 bit addresses.
- Slow access to data and quick code execution.
- Far pointers are preferred for data but not for code.
- Code is limited to 64KB, while data has a 1MB range.
- All functions are near by default and all data pointers are far by default.

**Large**

- Both code and data are allowed to use multiple segments.
- All pointers are 32 bits in length.
- Code execution is slower.
- Far pointers are used for both code and data, specifying both a 1 MB range.
- All functions and data pointers are far by default.

**Huge**

- Both code and data are allowed to use multiple segments.
- Every pointer is of 32 bits in length.
- Code execution is the slowest.
- Far pointers are used for both code and data.

**Segment and Offset Address**

Every address has two parts, segment and offset. These address parts can be separated using two macros defined in dos.h header file.

FP\_SEG() – this macro is used to obtain segment address of the given pointer variable.

FP\_OFF() – this macro is used to obtain segment address of the given pointer variable.

**Program to obtain segment and offset address**

```
#include<dos.h>
#include<iostream.h>
#include<constream.h>
Void main()
{
Int ch;
Cout<<"\n complete address of ch:"<<&ch;
```

```

Cout<<"segment address:"<<hex<<FP_SEG(&ch);
Cout<<"offset address:"<<hex<<FP_OFF(&ch);
}

```

### OUTPUT

Complete address of ch: 0x8f3f4442

Segment address: 8f34

Offset address: fff2

### Define far, huge and near pointers.

Far pointer – it is a 32 bit pointer and contains both segment and offset address parts.

Huge pointer – it is a 32 bit pointer and contains both segment and offset address parts.

Near pointer – it is a 16 bit pointer and contains both segment and offset address parts.

### 9. Explain in detail about new and delete operators.

- The new operator not only creates an object but also allocates memory.
- The new operator allocates correct amount of memory from the heap that is also called as a free store.
- The object created and memory allocated by using new operator should be deleted by the delete operator otherwise such mismatch operations may corrupt the heap or may crash the system.
- The delete operator not only destroys object but also releases allocated memory.
- The new operator creates an object and it remains in the memory until it is released using delete operator. Sometimes the object deleted using the delete operator remains in memory.
- If we send a null pointer to delete operator, it is secure. Using delete to zero has no result.
- The statement delete x does not destroy the pointer x. it destroys the object associated with it.
- Do not apply C functions such as malloc(), realloc() or free() with new and delete operators.
- Do not destroy the pointer repetitively or more than one time. First time the object is destroyed and memory is release. If for the second time the same object is deleted, the object is sent to the destructor and it will corrupt the heap.
- If the object created is not deleted, it occupies the memory unnecessarily.

### Program 1

```

#include<iostream.h>
#include<conio.h>
void main()
{
clrscr();
int *p, i;

```

```

p = &i;
p = new int[3];
*p=2; // first element
*(p+1) = 3; // second element
*(p+2) = 4; // third element
cout<<" value address";
for(int x=0;x<3;x++)
cout<<endl<<*(p+x)<<"\t"<<(unsigned)(p
+x);

```

```

delete []p;
}

```

**OUTPUT**

| Value | Address |
|-------|---------|
| 2     | 3350    |
| 3     | 3352    |
| 4     | 3354    |

**Explanation:** In the above program integer variable i and pointer \*p are declared. The pointer p is initialized with address of variable I and using new operator memory for three integers is allocated to it. The pointer \*p can hold three integers in successive memory locations. The pointer variable is initialized with numerical values. The for loop is used to display the contents of the pointer \*p. the delete operator releases the memory.

**Program 2**

```

#include<iostream.h>
#include<conio.h>
struct boy
{
char *name;
int age;
};
void main()
{
clrscr();
boy *p;
p = new boy[2];
p → name = "Rahul";
p → age = "20;
(p+1) → name = "Raj";

```

```

(p+1) → age = 21;
for(int x = 0; x<2; x++)
{
cout<<"\n                                Name:
"<<(p+x)→name<<endl<<"Age:"<<(p+x)
→age;
}
delete []p;
}

```

**OUTPUT**

```

Name : Rahul
Age: 20
Name: Raj
Age: 21

```

**10. Explain in detail about overloading new and delete operator?**

The memory allocation operators new and delete can be overloaded to handle memory resource in a customized way. It allows the programmer to gain full control over the memory resource and to handle resource crunch error such as out of memory, within a class. The main reason for overloading these function is to increase the efficiency of memory management. An application designed to handle memory allocation by itself through overloading can easily detect memory leaks(improper usage). It can also be used to create the illusion of infinite amount of main memory.

The normal call to a new operator , such as

ptr=new vector;

Dynamically creates a vector object and return a pointer to that object. The overloaded operator function new in the vector class not only creates an object, but also allocates the resources for its internal data members.

**//resource.cpp**

```
#include<iostream.h>
const int array_size=10;
class vector
{
private:
int *array;
public:
//overloading new operator
void * operator new(size_t size)
{
vector *my_vector;
my_vector=::new vector; //it refers to
global new, otherwise leads to recursive
call of vector::new
my_vector->array=new int [array_size];
//calls ::new
return my_vector;
}
```

**// overloading of delete operator**

```
void operator delete(void * vec)
{
vector *my_vect;
ny_vect=(vector *)vec;
delete(int *)my_vect->array;
::delete vec; //it refers to global delete,
otherwise leads to recursive call of
vector::delete
}
void read();
int sum();
};
void vector::read()
{
for(int i=0;i<array_size;i++)
```

```
{
cout<<"vector["<<i<<"]="?";
cin>>array[i];
}
}
int vector::sum()
{
int sum=0;
for(int i=0;i<array_size;i++)
sum+=array[i];
return sum;
}
void main()
{
vector *my_vector=new vector;
cout<<"enter vector data.."<<endl;
my_vector->read();
cout<<"sum of
vector="<<my_vector->sum();
delete my_vector;
}
```

### **OUTPUT**

```
Enter vector data..
Vector[0]=? 1
Vector[1]=? 2
Vector[2]=? 3
Vector[3]=? 4
Vector[4]=? 5
Vector[5]=? 6
Vector[6]=? 7
Vector[7]=? 8
Vector[8]=? 9
Vector[9]=? 10
Sum of vector=55
```

Overloaded **new** and **delete** operators function within the class and always static. We know that static function can be invoked without specifying object. Hence, **this** pointer is absent in the body of static functions. The compiler invokes the overloaded new operator and allocates memory before executing constructor. The compiler invokes the overloaded delete operator function and deallocates memory after execution of destructor.

**// program to display the sequence of execution of constructors and destructors in class**

```
#include<iostream.h>
#include<string.h>
#include<stdlib.h>
#include<conio.h>
class boy
{
char name[10];
public:
void *operator new(size_t);
void operator delete(void *q);
boy();
~boy();
};
char limit[sizeof(boy)];
boy::boy()
{
cout<<endl<<"in constructor";
}
boy::~~boy()
{
cout<<endl<<"in destructor";
}

void *boy::operator new(size_t s)
{
cout<<endl<<"in boy::new operator";
return limit;
}
void *boy::operator delete(void *q)
{
cout<<endl<<"in boy::delete operator";
}

void main()
{
clrscr();
boy *e1;
e1=new boy;
delete e1;
}
```

### Output

```
In boy::new operator
In constructor
In destructor
In boy:: delete operator
```

**Explanation:** In this program, the new and delete operators are overloaded. The class also has constructor and destructor. The overloaded new operator function is executed first followed by class constructor. We know that constructor is always used to initialize members. The constructor also allocates memory by calling new operator implicitly. Here, the new operator is overloaded. As soon as control of program reaches to constructor it invokes overloaded new operator before executing any statement in constructor. Similarly, when object goes out of scope destructor is executed. The destructor invokes overloaded delete operator to release the memory allocated by new operator.

|                                                                    |
|--------------------------------------------------------------------|
| <b>12. Explain the concept of specifying address of an object.</b> |
|--------------------------------------------------------------------|

The compiler assigns address to objects created. The programmer has no power over address of the object. However, it is possible to create objects at memory location given by the program explicitly. The address specified must be big enough to hold the object. The object created in this way must be destroyed by invoking destructor explicitly. The following program clears this concept.

**//program to create object at given address**

```
#include<iostream.h>
#include<conio.h>
class data
{
int j;
public:
data(int k)
{
j=k;
}
void *operator new(size_t,void *u)
{
return (data *)u;
}
void show()
{
cout<<"j="<<j;
```

```
}};
void main()
{
clrscr();
void *add;
add=(void*)0x420;
data *t=new (add) data(10);
cout<<"\n address of object:"<<t;
cout<<endl;
t->show();
t->data::~~data();
}

Output
Address of object: 0x8f800420
j=10
```

**Explanation:** In the above program, the operator new is overloaded. The void pointer \*add is declared and initialized with address 0x420. The pointer object \*p is declared and address is assigned to it. The new operator is also invoked to allocate memory. In the same statement constructor is also invoked and a value is passed to it. The member function show() displays the value of a member variable. Finally, the statement t->data::~~data(); invokes the destructor to destroy the object. We can confirm the address of object by displaying it. The address of object would be same as the specified one.

### 13. Explain the dynamic objects.

C++ supports dynamic memory allocation. C++ allocates memory and initializes the member variables. An object can be created at run-time. Such object is called as dynamic object. The constructor and destructor of dynamic object is explicitly done by the programmer. The dynamic objects can be created and destroyed by the programmer. The operator new and delete are used to allocate and deallocate memory to such objects. A dynamic object can be created using new operator as follows

```
ptr=new classname;
```

The new operator returns the address of object created and it is stored in the pointer ptr. The variable ptr is a pointer object of the same class. The member variables of object can be accessed using pointer and →(arrow) operator. A dynamic object can be destroyed using delete operator follows:

```
delete ptr;
```

It destroys the object pointed by pointer ptr. It also invokes the destructor of a class. The following program explains creation and deletion of dynamic object.

//program to create dynamic object

```
#include<iostream.h>
#include<conio.h>
class data
{
int x,y;
public:
data()
{
cout<<"\n constructor";
x=10;
y=50;
}
~data()
{
cout<<"\n destructor";
}
void display()
{
cout<<"\n x="<<x;
```

```
cout<<"\ny="<<y;
}
};
void main()
{
clrscr();
data *d;
d=new data;
d→display();
delete d;
}
```

#### Output

```
Constructor
X=10
Y=50
Destructor
```

**Explanation:** The d is a pointer object. The statement d= new data creates an anonymous object and assigns its address to pointer d. such creation of object is called as dynamic object. The constructor is executed when dynamic object is created. The statement

d->display() invokes the member function and contents of members are displayed. The statement delete d destroys the object by releasing memory and invokes destructor.

#### 14. Explain in detail about Heap consumption

- Heap is used to allocate memory during program execution (run-time). In assembly language, there is no such thing as heap. All memory is for programmer can use it directly.
- The heap is the piece of memory that we allocate. Local variables are stored in the stack and code is in code space.
- The local variables are destroyed when a function returns. Global variables are stored in the data area. As started earlier all local variables are stored in the stack.
- As soon as the function execution completes, local variables are destroyed and stack becomes empty.
- The heap is not cleared until program execution completes. It is user task to free the memory. The memory allocated from heap remains available until the user explicitly deallocated it.
- In C, the function, malloc() is used to allocate memory and if this function fails to allocate memory, returns null pointer.
- C++ allows us to apply the same logic with new operator. C++ allows us to use two function pointers known as **\_new\_handler** and **set\_new\_handler**.
- The \_new\_handler holds a pointer to a function. It requires no arguments and returns void.

##### //program to use set\_new\_handler

```
#include<iostream.h>
#include<new.h>
#include<stdlib.h>
#include<conio.h>
void m_warn()
{
    cerr<<"Cannot allocate!";
    exit(1);
}
int main()
{
    clrscr();
    set_new_handler(m_warn);
```

```
char *k=new char[50];
cout<<"\n First
allocation:k"<<hex<<long(k);
k=new char[64000u];
cout<<"\second
allocation:k="<<hex<<long(k);
set_new_handler(0);
return 0;
}
```

##### Output

First allocation: k=8fa40d48  
Cannot allocate!

**Explanation:** In the above program, the set\_new\_handler is initialized with the function m\_warn(). The m\_warn() displays warning message. The variable p is a character pointer and by using new operator, memory for 50 characters is allocated to the pointer p. the cout statement plays the starting address of the memory allocated.

Consider the statement p=new char [64000U]. In this statement the new operator attempts to allocate memory to pointer p. in case the new operator fails to allocate the memory, it calls the set\_new\_handler and m\_warn() function is executed.

### 15. What are virtual functions? Explain with a suitable program.

Virtual functions of base classes must be redefined in the derived classes. The programmer can define a virtual function in a base class and can use the same function name in any derived class, even if the number and type of arguments are matching. The matching function overrides the base class function of the same name. Virtual functions can only be member functions. We can also declare the functions as given below:

int Base::get(int) and int Derived::get(int) even when they are not virtual.

The base class version is available to derived class objects via scope override. If they are virtual, only the function associated with the actual type of objects is available. With virtual function we cannot alter the function type. If two functions with the same name have different arguments, C++ compiler considers them as different, and the virtual function mechanism is dropped.

**Program to declare virtual function and execute the same function defined in base and derived classes.**

```
#include<iostream.h>
#include<conio.h>
class first
{
    int b;
    public:
    first() { b = 10;}
    virtual void display() { cout<< "\n b = "
    <<b;}
};
class second : public first
{
    int d;
    public:
    second() { d = 20;}
```

|                                       |                 |
|---------------------------------------|-----------------|
| void display() { cout<<"\n d = "<<d;} | p -> display(); |
| };                                    | p = &s;         |
| int main()                            | p -> display(); |
| {                                     | return 0;       |
| clrscr();                             | }               |
| first f, *p;                          | <b>OUTPUT</b>   |
| second s;                             | b = 10          |
| p = &f;                               | d = 20          |

**Explanation**

The above program is the same as the previous one. The only difference is that the virtual keyword precedes the display() function of the base class as per the statement virtual void display() { cout << "\n b = " <<b;}. The virtual keyword does the runtime binding. In first call, the display() function of base class is executed and in second call after assigning address of derived class to pointer p.display() function of derived class is executed.

|                                                     |
|-----------------------------------------------------|
| <b>16. Explain the concept of Array of pointers</b> |
|-----------------------------------------------------|

Polymorphism refers to late or dynamic binding, i.e., selection of an entity is decided at run-time. Dynamic binding is associated with object pointers. Thus, address of different objects can be stored in any array to invoke function dynamically. The following program explains this concept.

**// program to create an array of pointers. Invoke functions using array objects.**

|                      |                   |
|----------------------|-------------------|
| #include<iostream.h> | void show()       |
| #include<conio.h>    | {                 |
| class A              | cout<<"B\n";      |
| {                    | }};               |
| public:              | class C:public A  |
| virtual void show()  | {                 |
| {                    | public:           |
| cout<<"A\n";         | void show()       |
| }                    | {                 |
| };                   | cout<<"C\n";      |
| class B: public A    | }                 |
| {                    | };                |
| public:              | class D: public A |

```

{
public:
void show()
{
cout<<"D\n";
}
};
class E:public A
{
public:
void show()
{
cout<<"E";
}
};
void main()
{
clrscr();
A a;
B b;
C c;
D d;
E e;
A *pa[]={ &a,&b,&c,&d,&e};
for(int j=0;j<5;j++)
pa[j]→show();
}

```

**Output**

```

A
B
C
D
E

```

**Explanation:** In the above program, class A is a base class. The classes B,C,D and E are derived class A. all these classes have a similar function show(). In function main(), a,b,c,d and e are objects of classes A,B,C,D and E respectively. The function show() of base class is declared virtual. An array of pointer \*pa is declared and it is initialized with the address of base and derived the classobjects. Using for loop and array, each object invokes function show(). If the base class function show() is non-virtual then every time function show() of base class is executed.

**17. Explain in detail about Abstract class**

Abstract classes are likely skeleton upon which new classes are designed to assemble well-designed class hierarchy. The set of well-tested abstract classes can be used and the programmer only extends them. Abstract classes containing virtual function can be used as help in program debugging. When various programming work on the same project, it is necessary to create a common abstract base class for them.

**//example program**

```

#include<iostream.h>
#include<conio.h>
class A
{
virtual void show()
{
cout<<"No function show() defined for
this calss";
}};
class B:public A
{
int b;

```

```

public:
B(int j=0)
{
b=j;
}
void show()
{
cout<<"\n In derived lassb="<<b;
}
};
class C: public A
{
int c;
public:
B(int k)
{
c=k;
}
};
void main()
{
clrscr();
B a(1);
C b(5);
a.show();
b.s how();
}

```

**Output**

In derived class b=1

No function show() defined for this class

**Explanation:**

Observe the contents of the file abstract.h. The structure debug contains virtual function show(). This file is #include in the above program. Classes A and B are derived from class debug which is defined in header file abstract.h. In function, a and b are objects of class A and B respectively. The statement a.show() invokes the function show() and value of a is displayed. The object b also invokes the function show(). However, the class B does not have function show(). Hence, function show() of abstract base class is executed which displays a warning message. An abstract class develops into a dominant and powerful interface when software system undergoes various changes. It is essential to confirm that the debugging interface is accurately constructed. If changes are made in actual project, it is compulsory to add appropriate methods to abstract base classes. In case the programmer needs to define a function warn() to the class debug, then the header file can be updated. The contents of file would be

**Contents of abstract.h header file**

```

#include<iostream.h>
struct debug
{
virtual void show()
{
cout<<"\n No function show() defined for this class";
}
};
virtual void warn()
{
cout<<"\n No function warn() defined for this class";
}

```

```
};
```

While defining such abstract classes, following points must be followed:

- Do not declare objects of abstract class type
- An abstract class can be used as a base class
- The derived class should not have pure virtual functions. Objects of derived classes can be declare.

### 18. Explain the concept of Virtual function in derived classes

We know that functions are declared virtual in base classes, it is mandatory to reduce virtual functions in the derived class. The compiler creates VTABLES for the derived classes and stores address of function in it. In case the virtual function is not redefined in the derived class, the VTABLE of derived class contain address of base class virtual function. Thus, the VTABLE contain the address of all functions.

#### //example program

```
#include<iostream.h>
#include<conio.h>
class A
{
public:
virtual void joy()
{
cout<<endl<<"In joy of class A";
}
};
class B:public A
{
public:
void joy()
{
cout<< endl<<"In joy of class B";
}
void virtual joy2()
{
cout<< endl<<"In joy2 of class B";
}
}
```

```
}
};

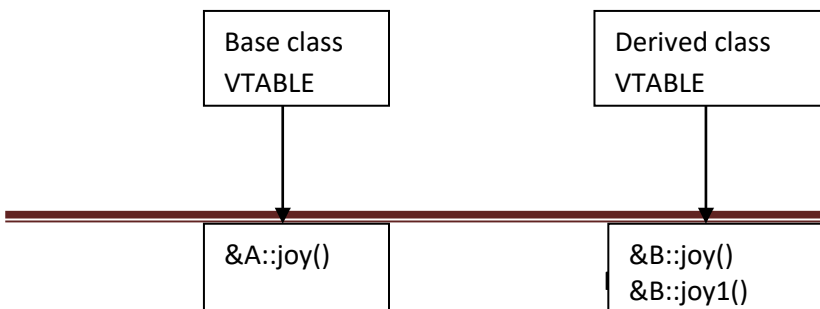
void main()
{
clrscr();
A *a1,*a2;
A a3;
B b;
a1=&a3;
a2=&b;
a1->joy();
a2->joy();
//a2->joy2(); //joy2 is not member of class A
}
```

#### OUTPUT

```
In joy of class A
In joy of class B
```

#### Explanation:

In the above program, the base class A contains one virtual function joy(). In the derived class B, the function joy() is redefined and defines a new virtual function joy2(). VTABLES created for base and derived classes.



a2 → joy2();

the above statement will generate an error message. In the above statement, pointer a2 is treated as only pointer to base class object. The function joy2() is not a member of base class A. hence it is not allowed to invoke the function joy2() using the pointer object a2. However, an address of derived class is assigned to base class pointer, and the compiler in no way can determine that we are working with derived class objects. The compiler avoids calls to virtual functions present only in derived classes. In hierarchical class organization of various levels, if it is essential to invoke a function at any lever by using base class pointer, then the function should be declared virtual in the base class.

### 19.Ellobrate the concept of constructors and virtual functions

It is possible to invoke a virtual function using a constructor. Constructor makes the virtual mechanism illegal. When a virtual function is invoked through a constructor, the base class virtual function will not be called and instead of this the member function of the same class is invoked.

#### //program to call virtual function

```
through constructors
#include<iostream.h>
#include<conio.h>
class B
{
int k;
public:
B(int j)
{
k=j;
}
virtual void show()
{
cout<<endl<<"k="<<k;
}
};
class D:public B
{
```

```
int h;
public:
D(int m,int n):B(m)
{
h=n;
B *b;
b=this;
b→show();
}
void show()
{
cout<<endl<<"h="<<h;
}
};
void main()
{
clrscr();
B b(4);
D d(5,2);
```

```
}
```

### Output

```
h=2
```

**Explanation:** in the above program, the base class B contains a virtual function. In the derived class D the same function is redefined. Both the base and derived classes contain constructors. In the derived class constructor, a base class pointer \*b is declared. We know that this pointer contains address of object calling the member function. Here, the this pointer holds address of object d. the pointer object b invokes function show(). The derived class show() function is invoked. Here, though the object d is not fully constructed it still invokes the member function of the same class. This is possible because a virtual functions call reaches ahead into inheritance.

## 20. Give in detail about virtual destructors.

We know how virtual functions are declared. Likewise, destructor can be declared as virtual. The constructor cannot be virtual, but the virtual destructor is implemented in the way like virtual functions. Destructors of derived and base classes are called when a derived class object addressed by the base class pointer is declared.

**//program to define virtual destructor**

```
#include<iostream.h>
#include<conio.h>
class A
{
public;
A()
{
cout<<endl<<"In constructor of class A";
}
virtual ~A()
{
cout<<endl<<"In destructor of class A";
}
};
class D:public A
{
public;
D()
{
cout<<endl<<"In constructor of class D";
}
~D()
{
cout<<endl<<"In destructor of class D";
}
};

void main()
{
clrscr();
A *p;
p=new D;
delete p;
}
```

### Output

```
In constructor of class A
In constructor of class D
In destructor of class D
In destructor of class A
```

**Explanation:** in the above program, the destructor of the base class B is declared as virtual. A dynamic object is created is created and the address of nameless object created is assigned

to pointer p. the new operator allocates memory required for data members. When object goes out of scope it must be deleted and the same performed by the statement delete p. when derived class object is pointed by the base class pointer object, in order to invoke base class destructor, virtual destructors are useful.

### 21.Explain the Destructor and virtual functions.

When a virtual is invoked through a non-virtual member function, late binding is performed. When call to virtual function is made through the destructor, the redefined function of the same class is invoked. Consider the following program:

**//program to call virtual function using destructors**

```
#include<iostream.h>
#include<conio.h>
class A
{
public;
~A()
{
cout<<endl<<"Virtual destructor";
}
virtual void joy()
{
cout<<endl<<"In joy of class A";
}
};
class D:public A
{
public;
~D()
{

```

```

{
A *p;
p=this;
p->joy();
}
void joy()
{
cout<<endl<<"In joy of class D";
}
};
void main()
{
clrscr();
D x;
}

```

#### **Output**

In joy of class D

In virtual constructor

**Explanation:** in the above program, in destructor of the derived class function joy() is invoked. The member function joy() of derived class is invoked followed by virtual destructor.

## **STRING**

### **Write in detail about Handling string objects**

The member functions insert(),replace(),erase() and append() are used to modify string contents. The following program illustrates the use of these functions:

**// program to insert, replace, erase and append.**

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
string s1("abchijk");
string s2("defg");
string s3("abc12345defg");
string s4("abcdefg");
string s5("abcd");
string s6("ABCD");
cout<<\n Before insertion";
cout<<"\n s1="<<s1;
cout<<"\n s2="<<s2;
cout<<"\n After insertion";
s1.insert(3,s2);
cout<<"\n s1="<<s1;
cout<<"\n s2="<<s2;

s3.erase(3,5);
cout<<"\n String(erase):"<<s3;
s4.replace(1,3,"BCD");
cout<<"\n Replaced string:"<<s4;
s5.append(s6);
cout<<"\n Appended string:"<<s5;
return 0;
}
```

**Output**

**Before insertion**

**S1= abchijk**

**S2= defg**

**After insertion**

**S1= abcdefghijk**

**S2= defg**

**String(erase):abcdefg**

**Replaced string:aBCDefg**

**Appended string: abcdABCD**

**Explain Relational operators using string.**

### String manipulation operators

| operator | Working                      |
|----------|------------------------------|
| =        | Assignment                   |
| +        | Joining two or more string   |
| +=       | Concatenating and assignmeny |
| ==       | Equality                     |
| !=       | Not equal to                 |
| <        | Less than                    |
| >        | Greater than                 |

|    |                               |
|----|-------------------------------|
| <= | Less than or equal            |
| >= | Greater than or equal         |
| [] | Subscription(used with array) |
| << | Insertion operator            |
| >> | Extraction operator           |

**//usage of compare(),equality operator**

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
string s1("OOP");
string s2("OOP");
string s3("aaa");
string s4("bbb");
int d;
if(s1==s2)
{
cout<<"\n s1 and s2 strings are identical";
else
cout<<"\n s1 and s2 strings are different";
}
```

```
d=s1.compare(s2);
if(d==0)
{
cout<<"\n Both strings are identical";
else if(d>0)
cout<<"\n s3 is greater than s4";
else
cout<<"s4 is greater than s1";
}
return 0;
}
```

**Output**

**s1 and s2 strings are identical**

**s4 is greater than s3**

**Explain the String attributes**

The various attributes of the string such as size,length,capacity etc. can be obtained using member function.

| function   | use                                                              |
|------------|------------------------------------------------------------------|
| size()     | Provides no. of character in the string                          |
| length()   | Calculate total no. of elements in the string                    |
| capacity() | Calculates the total elements that can be stored                 |
| max_size() | Calculates the maximum possible size of string in a given system |
| empty()    | Returns false if the string is not empty, otherwise true         |

// program to size,length,capacity,

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
string s1;
cout<<"\n Size if s1 before initialization\n size()\t\t length() \t\t capacity \t\t max_size()";
cout<<"\n\t\t"<<s1.size()<<"\t \t"<<s1.length()<<"\t\t"<<s1.capacity()<<"\t\t"<<s1.max_size();
s1="abcdefghijklmnopqrstuvwxyabcdefghijklmnop";
cout<<"\n Size if s1 after initialization\n size()\t\t length() \t\t capacity";
cout<<"\n\t\t"<<s1.size()<<"\t \t"<< s1.length()<<"\t\t"<<s1.capacity();
return 0;
}
```

### Output

**Size if s1 before initialization**

| size() | length() | capacity | max_size   |
|--------|----------|----------|------------|
| 0      | 0        | 0        | 4294967293 |

**Size if s1 after initialization**

| size() | length() | capacity |
|--------|----------|----------|
| 32     | 3263     |          |

### Explanation:

Size() and length() is used to displays the total no. of elements in the given string.

Capacity() is used to displays capacity of string to stored. If the string object is empty the function returns 0. In case the string object less than 32 characters, it returns 31. If the string is initialized with 32 characters, the capacity() returns the value 63. i.e,32+31.

**Explain the Accessing elements of strings**

It is possible to access particular word or single character of string with help of member functions of string class.

| function | use                                              |
|----------|--------------------------------------------------|
| at()     | Access the character located at given location   |
| begin()  | Returns a reference to the beginning of a string |
| end()    | Returns a reference to the end of a string       |
| find()   | Find the given sub string in the source string   |

//program to use at(),find(),find\_first-of(),find\_last-of().

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1("program");
    string s2("welcome to c++");
    string s3("c plus plus");
    for(int i=0;i<s1.length();i++)
    cout<<"\n Given string s1:"<<s1.at(i);//equals to s1[i] in array concept
    cout<<"\n Sub-string of s2:"<<s2.substr(11,3);
    cout<<"\n First occurrence of p:"<<s3.find_first_of('p');
    cout<<"\n Last occurrence of p:"<<s3.find_last_of('p');
    return 0;
}
```

### Output

**Given string s1: program**

**Sub-string of s2: c++**

**First occurrence of p: 2**

**Last occurrence of p: 7**

### Explanation:

In the above program,s1,s2 and s3 are string objects created and initialized. Using for loop the strings present in s1 are retrieved and displayed using the statement. The substr() is used to find the sub-string in the s2 string. It is used in following format:

```
s2.substr(s,e);
```

here s and e are integer variables that indicate the starting and ending element of the sub-string.

Hence s2.substr(11,3); displays the substring “c++”.

Next , the find\_first-of() searches the character ‘p’ , diaplays its first occurrence of the given string.

The find\_lasst-of() searches the character ‘p’ , diaplays its last occurrence of the given string.

### Explain the Compare and exchange operations using string.

The string class contains functions, which allows the programmer to compare and exchange the strings.

| function  | Use                                           |
|-----------|-----------------------------------------------|
| compare() | Compare two strings                           |
| swap()    | Exchange the given string with another string |

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
string s1(“Take”);
string s2(“Taken”);
int d=s1.compare(s2);
cout<<“\n d=“<<d;
d=s1.compare(0,4,s2,0,4);
cout<<“\n d=“<<d;
return 0;
}
```

### Output:

**d=-1**

**d=0**

### Explanation:

In the above program, s1 and s2 are two string objects initialized with the string “Take” and “Taken” respectively. The first compare() statement checks the two string objects. It returns -1, i.e., the strings are not identical. In the second compare() statement, element numbers of both the strings that are to be compared are sent. Thus, passing element number sub-string can be compared. In the above program 0 to 4 elements of both strings are compared. The function compare() returns 0. i.e., the two sub-strings are identical.

### //program to swap the given string

```
#include<iostream>
#include<string>
```

```
using namespace std;
int main()
{
    string s1("Take");
    string s2("Took");
    cout<<\n Before swap";
    cout<<"\n s1="<<s1;
    cout<<"\n s2="<<s2;
    cout<<"\n After swap";
    s1.swap(s2);
    cout<<"\n s1="<<s1;
    cout<<"\n s2="<<s2;
}
```

**Output:**

```
Before swap
s1=Take
s2=Took
Afterswap
s1=Took
s2=Take
```

**Explanation:**

In the above program, s1 and s2 are declared and initialized with "Take" and "Took". The swap() function exchanges the content of s1 and s2. i.e., contents of s1 is assigned to s2 and vice versa. The output of the program is given above.

**Give in detail Miscellaneous functions**

assign(): this function is used to assign wholly/partly to other string object. It is used in the following format:

```
s2.assign(s1);
```

in the above statement, the string s1 are assigned to s2.

//usage of assign()

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1("C plus plus");
    string s2;
    int x=0;
    s2.assign(s1,0,6);
    cout<<s2;
    return 0;
}
```

**Output****C plus****Explanation:**

In the above program,s1 and s2 are two string object. The object s1 is initialized with the value "C PLUS PLUS". The object s2 invokes the assign() function with the integer argument 0,6 that indicates the sub-string. The sub-string is assigned to object s2 i.e., "C plus"

**//program to find starting and ending character of a string**

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1("C PLUS PLUS");
    char *x,*y;
    x=s1.begin();
    cout<<"Starting character of given string:"<<*x;
    y=s1.end();
    cout<<"\nEnding character of given string:"<<*y;
}
```

**Output****Starting character of given string: C****Ending character of given string: S**

**Explanation:**

In the above program for the string class s1 object is created and initialized with the value “C PLUS PLUS” and “x,y” is a character pointer. The s1 invokes the function begin() and returns the starting address of the string s1 to character pointer x. The s1 invokes the function end() and returns the starting address of the string s1to character pointer y .